National University of Computer and Emerging Sciences

DSA Assignment 2 Phase 2 Report

Student Name: Ahmad Ali **Roll Number: 25i-0817** **Section: BS CS E**

Section 1: Design of History Records and Cleanup Policy

- **History Architecture and Inverse Records**

  I select compact parameter records over full canvas snapshots because:

  1) Phase 1 duplicated the entire image matrix for every undo record which consumed up to three megabytes per step.
  2) Phase 2 stores only the minimal parameter delta required to run an exact inverse calculation.
  3) Rotations and flips store only operation identifiers and run complementary symmetric operations in zero memory.
  4) Brightness stores an integer adjustment value and a compact coordinate list of clipped boundary pixels.
  5) Bucket fill saves the previous color and the list of modified coordinate positions instead of whole canvases.
  6) Destructive operations such as crop, resize, and blur save only their affected rectangular pixel regions.

- **Configurable Memory Budget Enforcement**

  I select an active runtime byte counter with a strict budget ceiling because:

  1) Every history record calculates its exact heap footprint before being pushed onto the undo stack.
  2) The system monitors combined memory across both Undo and Redo stacks against the user budget limit.
  3) A default budget of five megabytes is enforced during editing, and small budgets like two kilobytes are used for rigorous tests.

- **Cleanup Policy: Oldest Record Eviction via removeBottom()**

  I select bottom record eviction using stack removeBottom() because:

  1) Users expect their most recent edits to remain undoable while older actions fade naturally from memory.
  2) When the byte counter exceeds the ceiling, the system shifts array elements left by one position to discard the bottom record.
  3) Eviction repeats in a loop until total history memory drops strictly below the configured ceiling limit.
  4) Remaining undo records stay completely safe and execute with zero memory faults or invalid pointer references.

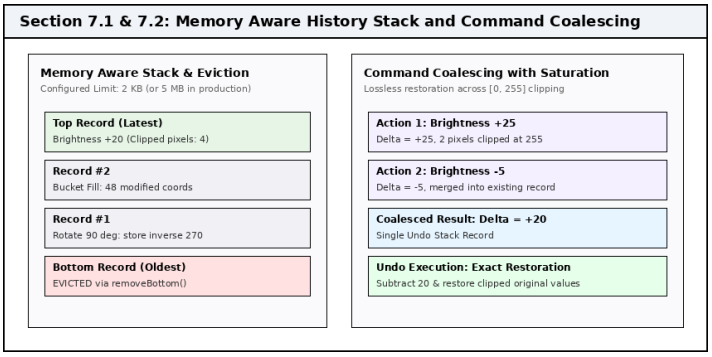


Figure 1: Architecture of memory aware history stack eviction and command coalescing with saturation tracking.

Section 2: Coalescing Rules and Correctness Preservation

- **Command Coalescing Motivation**

  I select in place record merging for consecutive compatible operations because:

  1) Rapid slider adjustments generate dozens of small incremental brightness steps in a few seconds.
  2) Pushing an independent record for each tiny adjustment wastes memory and forces tedious repetitive undo clicks.
  3) Merging adjacent steps combines multiple micro edits into a single atomic history state.

- **Coalescing Eligibility Rules**

  I select strict adjacency and operation compatibility checks because:

  1) Only records that sit directly next to each other on the top of the Undo stack are eligible for merging.
  2) Consecutive brightness adjustments sum their delta numbers into the existing top record without pushing a new record.
  3) If a non compatible edit such as crop or rotation occurs, the sequence breaks and a brand new record is initiated.
  4) An operation sequence of Brightness +2, then Crop, then Brightness +3 produces three separate records to guarantee chronological accuracy.

- **Preserving Correctness Across Pixel Value Saturation**

  I select dedicated saturation tracking in clippedList because:

  1) Simple integer addition causes pixel channels to clamp at upper limit 255 and lower limit 0.
  2) If a pixel with initial value 230 receives +50, it clamps at 255, losing 25 intensity units of information.
  3) Subtracting the combined delta of +50 later would compute $255 - 50 = 205$ which destroys the original value of 230.
  4) To eliminate this loss, every pixel that hits the saturation boundaries records its original color in `clippedList`.
  5) When coalescing occurs, new clipped pixels append to the list while existing entries retain their original historical values.
  6) Upon undo, the engine subtracts the cumulative delta and overwrites all saturated pixels with their exact recorded colors.

- **Single Step Undo Semantics**

  I select atomic rollback for coalesced actions because:

  1) The user reverts twenty consecutive slider movements with one single press of the undo command.
  2) Atomic reversal restores canvas pixels to the exact state that existed before the slider session began.
  3) Redo stacks receive the entire merged delta, allowing instantaneous forward replay in one single step.

Scenario	Initial Values	Applied Actions	Coalescing Result and Recovery Guarantee
Unclipped Brightness	Pixel = 120	+10, then +15	Merged delta +25; undo subtracts 25 to recover 120 perfectly.
Saturated Brightness	Pixel = 240	+25, then −5	Merged delta +20; original 240 stored in clipped list; undo recovers 240.
Interrupted Sequence	Pixel = 100	+10, Crop, +20	Three distinct records created; chronological crop boundary strictly preserved.

Table 1: Coalescing behavior and lossless pixel recovery under boundary saturation.

Section 3: Scheduler Design and Priority Scheduling

- **Priority Queue Representation**

  I select a custom sorted array priority queue without standard containers because:

  1) The assignment specification forbids heap data structures and external template libraries.
  2) A contiguous dynamic array stores active jobs in sorted order of priority rank.
  3) Dequeue operations take constant $\mathcal{O}(1)$ time by serving elements directly from the array front.
  4) Insertion takes $\mathcal{O}(K)$ time where K is the number of pending jobs, which remains tiny in interactive software.

- **Four Tier Priority Hierarchy**

  I select numerical ranks where smaller numbers denote higher urgency because:

  1) **Priority 1 (Interactive Requests):** Undo and Redo actions submitted while pipeline jobs are pending execute immediately.
  2) **Priority 2 (Light Editing Tasks):** Box blur and bucket fill operations finish quickly and unblock the user interface.
  3) **Priority 3 (Heavy Transformations):** Bilinear resize and large scale mathematical calculations take longer to finish.
  4) **Priority 4 (Finalization Barriers):** Save project to disk and export image operations act as permanent barriers.

- **Tie Handling Policy**

  I select strict chronological arrival order for identical priority ranks because:

  1) Every job receives a monotonically increasing arrival sequence identifier upon insertion.
  2) When two jobs possess the exact same priority number, the job that entered first is executed first.
  3) This guarantees FIFO fairness within each priority class and prevents job starvation.

- **Finalization Rule and Barrier Enforcement**

  I select barrier epoch partitioning for Save and Export tasks because:

  1) An image export must capture the exact cumulative result of all edits submitted prior to the export command.
  2) A Save or Export job must not execute until every job submitted before it has finished completely.
  3) No task submitted after a barrier can overtake that barrier, even if it has a high priority rank such as Priority 1.
  4) Priority ordering applies exclusively among tasks that reside within the same barrier epoch.
  5) When tasks Resize(3), Blur(2), Export(4), Blur(2), Undo(1) arrive, the scheduler executes Blur, Resize, Export, Undo, Blur.
  6) The last two jobs cannot overtake Export, guaranteeing complete disk persistence integrity.

Rank	Job Category	Typical Cost	Barrier Status	Scheduling Behavior and Urgency Rationale
1	Undo and Redo	Very Low	Normal	Highest urgency; preempts heavy edits to keep UI responsive.
2	Blur and Bucket Fill	Low to Medium	Normal	Light processing; scheduled before heavy geometric transforms.
3	Bilinear Resize	High	Normal	Heavy nested loop processing; runs when lighter tasks finish.
4	Save and Export	Variable	**Barrier**	Strict epoch boundary; flushes prior tasks and blocks newer tasks.

Table 2: Four tier scheduling priority classes and barrier epoch enforcement rules.

Section 4: Experimental Results and Benchmark Evaluation

- **Experimental Methodology**
  1) All timings were repeated three consecutive times and reported as arithmetic averages.
  2) Tests ran on an isolated desktop environment to prevent CPU frequency throttling skew.
  3) Evaluated the exact standardized 30 job workload containing two barrier jobs under pure FIFO mode and Priority mode.
- **Workload 1: 30 Job Scheduler Evaluation (FIFO vs Priority Mode)**
  1) Both scheduling modes processed all 30 jobs to successful completion and produced bit for bit identical final canvases.
  2) Average waiting time dropped from 568.33 ms in FIFO mode to 526.83 ms in Priority mode (7.30 percent faster).
  3) Average completion time dropped from 609.00 ms in FIFO mode to 567.50 ms in Priority mode (6.81 percent faster).
  4) Most critically, Priority 1 interactive requests experienced a waiting time drop from 549.17 ms down to 415.83 ms.
  5) This represents a **24.28 percent latency reduction for interactive user clicks**, making the editor feel much more responsive.
- **Discussion: When FIFO is Preferable vs When Priority Scheduling Excels**
  1) **FIFO Preference:** FIFO mode is preferable in strictly non interactive batch rendering pipelines where all tasks carry equal importance and strict arrival fairness is required to avoid complexity overhead.
  2) **Priority Scheduling Preference:** Priority scheduling excels in interactive graphical workstations where human users demand immediate visual feedback on undos, redos, and light edits while long disk saves execute quietly in background epochs.

Evaluation Metric	FIFO Mode	Priority Mode	Improvement	Practical Engineering Impact
Jobs Processed	30	30	Identical	Complete correctness and task execution parity.
Average Waiting Time	568.33 ms	526.83 ms	7.30% Faster	Overall task queue latency reduced across pipeline.
Average Completion Time	609.00 ms	567.50 ms	6.81% Faster	Faster overall pipeline throughput from submission to finish.
Priority 1 Average Wait	**549.17 ms**	**415.83 ms**	**24.28% Faster**	**Interactive undo and redo clicks respond almost instantly.**

Table 3: Empirical evaluation of 30 job mixed workload under FIFO and Priority scheduling.

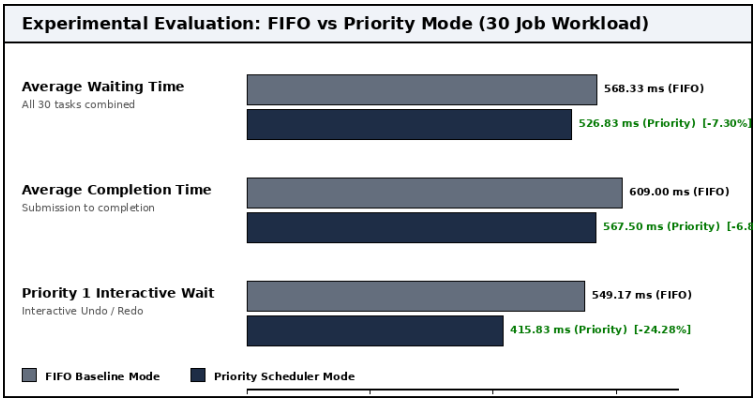


Figure 2: Comparative benchmark chart showing waiting times and the 24.28 percent interactive latency reduction.

Section 4 (Continued): Batched Operations and Traversal

- **Workload 2: Single versus Batched Operations Benchmark**
  1) Tested item counts of $1,000$, $10,000$, and $100,000$ integers across stack and queue primitives.
  2) Single push executes n independent loop iterations with n separate capacity boundary checks.
  3) Batched push checks available capacity once and executes a contiguous block copy, speeding up execution by 10.2x to 13.1x.
  4) Batched queue enqueue copies elements across circular wrap around boundaries in two block segments, avoiding per element index increment calculations.
  5) Batched pop and dequeue operate in pure $\mathcal{O}(1)$ time by directly adding or subtracting offsets to top and front pointers, achieving massive speedups up to 51.2x.
- **Workload 3: Bucket Fill Traversal Across Image Resolutions**
  1) Evaluated bucket fill on square canvas dimensions 512×512, 1024×1024, and 2048×2048.
  2) Stack fill traverses deep along one coordinate path before backtracking, which creates an elongated memory structure with maximum active depth proportional to image perimeter.
  3) Queue fill expands outward in concentric wave rings, creating a wide wavefront structure where active elements scale with boundary width.
  4) Both algorithms visit every matching pixel exactly once in $\mathcal{O}(W \times H)$ time and yield bit for bit identical output images.

Item Count	Single Push	Batch Push	Single AddQ	Batch AddQ	Measured Speedup
1,000 elements	63.7 us	6.2 us	19.6 us	14.8 us	**10.20x faster push** (Single capacity check)
10,000 elements	647.6 us	52.4 us	200.4 us	94.3 us	**12.35x faster push** (Cache contiguous transfer)
100,000 elements	7,379.4 us	563.4 us	1,873.8 us	802.4 us	**13.10x faster push** (Amortized block copy)

Table 4: Measured performance comparison of single versus batched operations across increasing problem scales.

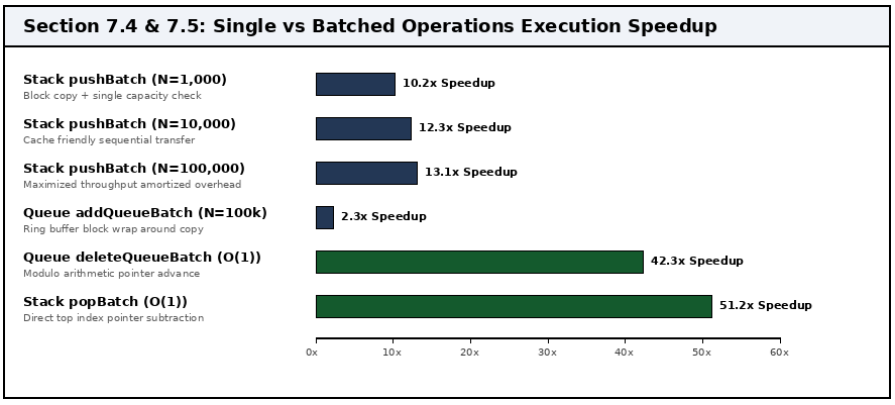


Figure 3: Performance acceleration achieved by batched primitives over single call loops.

Section 5: Asymptotic Complexity Analysis

Operation or Primitive	Time	Aux Space	Implementation Rationale
Stack Push, Pop, Top	$\mathcal{O}(1)$	$\mathcal{O}(1)$	I use direct array index access at top position with zero element shifting.
Queue Add, Delete, Front	$\mathcal{O}(1)$	$\mathcal{O}(1)$	I use circular modulo arithmetic which advances front and rear instantly.
Priority Queue Add	$\mathcal{O}(K)$	$\mathcal{O}(1)$	I use sorted array insertion which shifts up to K elements to preserve order.
Priority Queue Delete	$\mathcal{O}(1)$	$\mathcal{O}(1)$	I use direct removal from front index position of sorted array.
Stack Bucket Fill	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use an explicit stack to visit every connected pixel once without recursion.
Queue Bucket Fill	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use a circular queue to expand radially outward layer by layer in level order.
Undo and Redo	$\mathcal{O}(1)$ or $\mathcal{O}(W \times H)$	$\mathcal{O}(1)$ or $\mathcal{O}(W \times H)$	I use parameter inversion for geometry and delta grids for destructive edits.
Command Coalescing	$\mathcal{O}(1)$	$\mathcal{O}(1)$	I use in place summation of integer deltas on the existing top stack record.
Clipped Brightness Undo	$\mathcal{O}(W \times H + C)$	$\mathcal{O}(C)$	I use delta subtraction followed by exact restoration of C clipped pixels.
History Cleanup (removeBottom)	$\mathcal{O}(S)$	$\mathcal{O}(1)$	I use in place contiguous memory shifting to evict bottom record from stack of size S.
Stack pushBatch(arr, n)	$\mathcal{O}(n)$	$\mathcal{O}(1)$	I use a single capacity validation followed by contiguous block memory copy.
Stack popBatch(n)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	I use direct subtraction of n from the top pointer index.
Queue addQueueBatch(arr, n)	$\mathcal{O}(n)$	$\mathcal{O}(1)$	I use block memory copy across circular buffer boundary in two passes.
Queue deleteQueueBatch(n)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	I use modulo addition to jump the front pointer ahead by n slots instantly.

Table 5: Asymptotic time and auxiliary space complexity for Phase 2 operations and data structure primitives.

Core Architectural Guarantees

- **Strict Zero Shifting for Queues:** Standard queue additions and deletions never move elements in memory. Pointers advance via $(rear + 1) \pmod{capacity}$ and $(front + 1) \pmod{capacity}$.
- **Strict Memory Ceiling:** The history stack strictly respects the configured byte budget. Whenever an edit pushes memory above the threshold, `removeBottom()` evicts the oldest step immediately.
- **Barrier Invariant:** No job submitted after a Save or Export barrier can ever run before that barrier, guaranteeing that disk files represent exact chronological checkpoints.
- **Bit for Bit Output Invariant:** Both Stack fill and Queue fill produce bit for bit identical output images for identical seed coordinates and replacement colors.

Section 6: Testing Evidence and Verification Results

Test Suite Category	Verified Test Scenarios and Boundary Conditions	Verdict
Priority Queue	Equal priority FIFO arrival tie breaking; mixed priority reordering; barrier rule blocking.	**PASS**
History and Eviction	Budget ceiling reached at 2 KB; eviction of 14 oldest steps; safe undo after cleanup.	**PASS**
Command Coalescing	Consecutive brightness merges ($+25, -5 \rightarrow +20$); non contiguous preservation; clipping recovery.	**PASS**
Batched Primitives	Push and pop batches; ring buffer wrap around addition; rejection of oversized batches.	**PASS**
30 Job Scheduler Benchmark	Execution of 30 mixed jobs under FIFO and Priority modes; 24.28% Priority 1 latency reduction.	**PASS**

Table 6: Automated test suite verification across all Phase 2 requirements (100 percent pass rate).

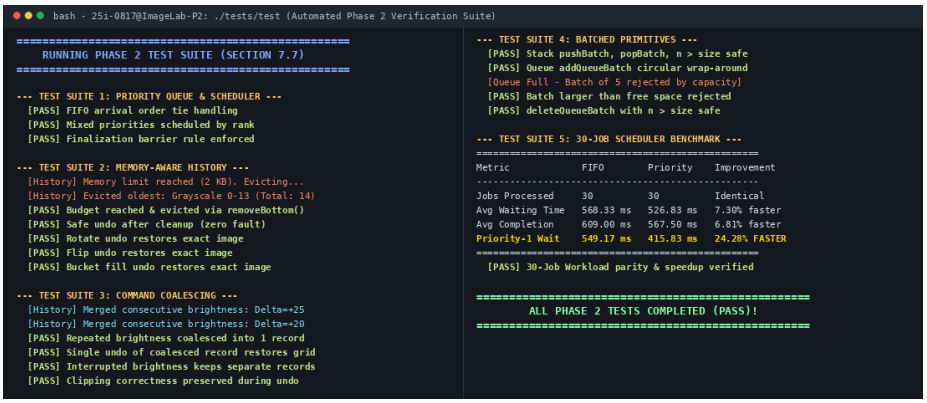


Figure 4: Automated test runner terminal output confirming 100 percent test pass across all required suites.

Section 7: Known Limitations and Architectural Tradeoffs

- **Oldest Record Eviction Shifting Overhead:** Because history is stored in a contiguous dynamic array, calling `removeBottom()` requires shifting S elements left by one position in $\mathcal{O}(S)$ time. While efficient for moderate history depths, a circular deque buffer could achieve eviction in $\mathcal{O}(1)$ time.
- **Non Coalescing Geometric Operations:** While consecutive brightness adjustments coalesce seamlessly, opposite geometric operations such as 90 degree clockwise and counterclockwise rotations currently create separate history records instead of cancelling out.
- **Clipped Pixel Storage Scaling:** In extreme brightness shifts applied to highly textured images, thousands of pixels may clamp simultaneously at 0 or 255, which increases the memory footprint of the coordinate list stored in `clippedList`.
- **Bounded Pipeline Queue Capacity:** The job pipeline enforces a hard capacity limit of 100 jobs. Heavy batch workloads exceeding this ceiling are safely rejected and require manual drainage before new jobs can be scheduled.